
Bài 2

Thiết kế truy vấn dữ liệu

Nội dung



1. Các bước viết truy vấn dữ liệu

2. Xây dựng điều kiện lọc dữ liệu

3. Gom nhóm dữ liệu và các hàm tập hợp

4. Truy vấn trên nhiều bảng dữ liệu

5. Truy vấn con (subquery)

Các yêu cầu khi xây dựng truy vấn dữ liệu

**Know
your
data**

Có bao nhiêu dữ liệu?
Dữ liệu nằm ở đâu?



**Ý nghĩa của các bảng dữ liệu
và liên kết**

**Pose
the
question**

Phân tích các phát biểu / yêu cầu
thông tin



Viết lại gần với “ngôn ngữ” database
Biểu diễn trực quan

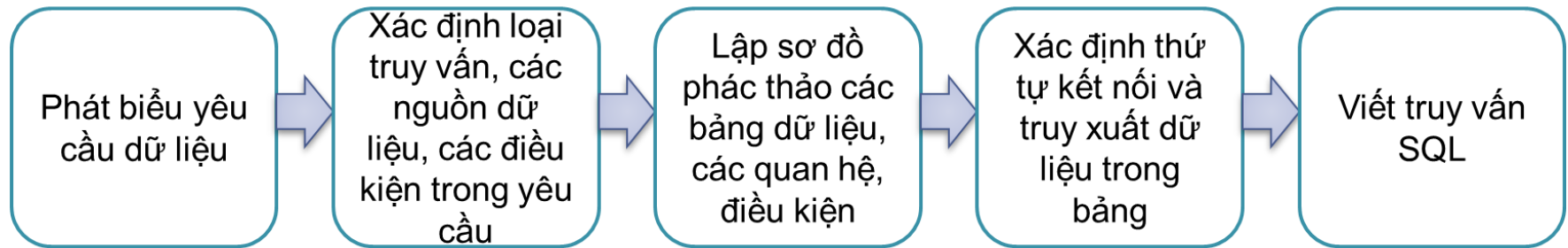
**Write
the
query**

Viết truy vấn SQL



SELECT, INSERT, UPDATE, DELETE
where, group by, order by,...

Các bước viết truy vấn



Các bước cơ bản thực hiện truy vấn

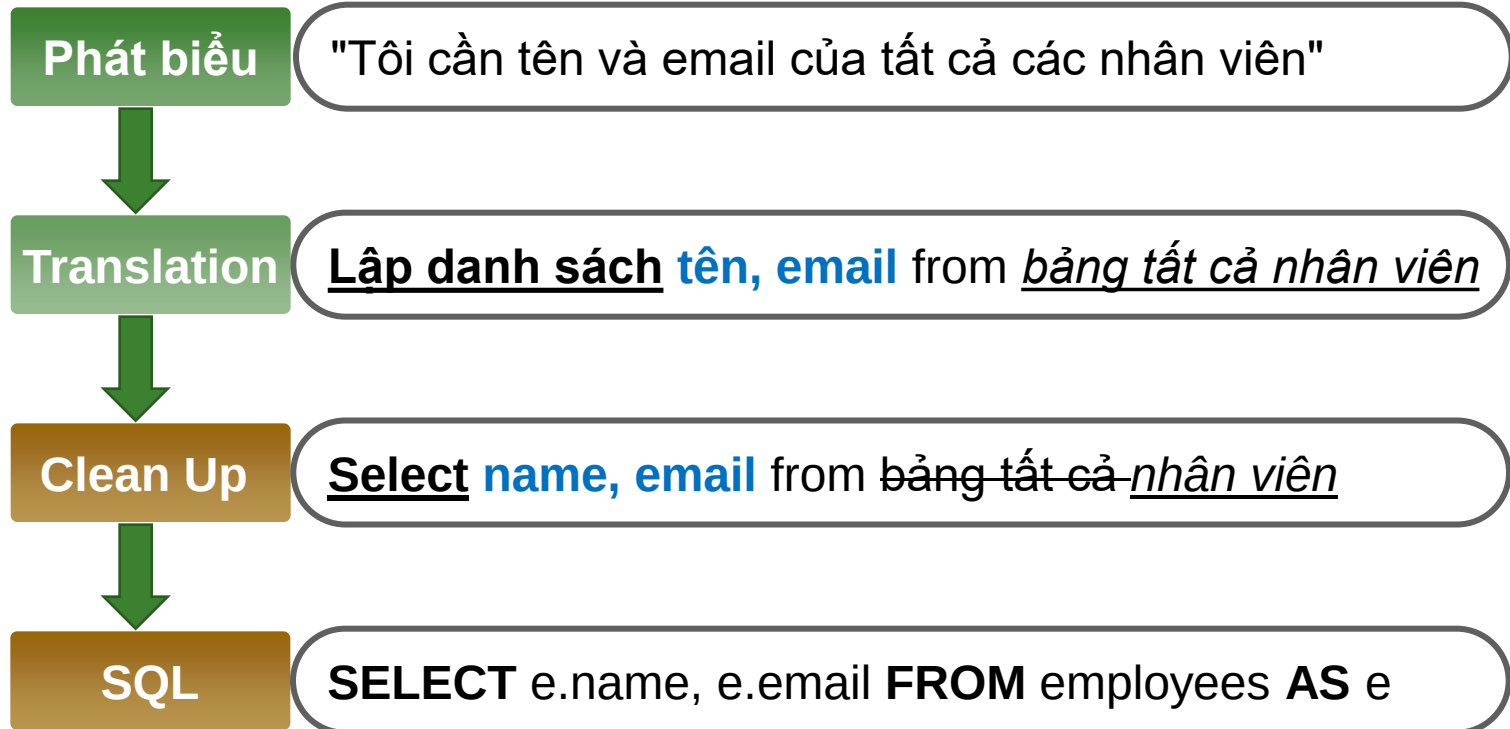
- Xác định (chọn) hình thức truy vấn sẽ thực hiện
- Xác định (chọn) bảng sẽ tham gia truy vấn.
- Xác định (chọn) các field sẽ tham gia truy vấn : FIELD đã có sẵn; field cần tạo mới, cách tạo field mới.
- Xác định việc sắp xếp dữ liệu trong field (nếu có)
- Xác định việc hiện hoặc ẩn field trên màn hình kết quả truy vấn.
- Xác định điều kiện chọn lọc dữ liệu cho các field
- Xem kiểm tra kết quả .
- Thực hiện (chạy) truy vấn → đóng truy vấn → Save & đặt tên cho truy vấn

Các bước viết truy vấn

- Viết lại các yêu cầu truy vấn gần với “ngôn ngữ” của database
 - Khi yêu cầu thông tin từ CSDL, các yêu cầu có thể dưới dạng câu hỏi hoặc một phát biểu
 - Ví dụ như:
 - Văn phòng của công ty đặt tại những quốc gia nào?
 - Lập danh sách nhân viên và email của họ.
 - Hiển thị tên của các nhân viên có tuổi và ngày bắt đầu làm việc trong công ty.
 - ...
 - Sau khi đã hiểu đúng về yêu cầu truy vấn, cần viết lại các yêu cầu này dưới dạng các phát biểu hình thức gần với “ngôn ngữ” của CSDL

Các bước viết truy vấn

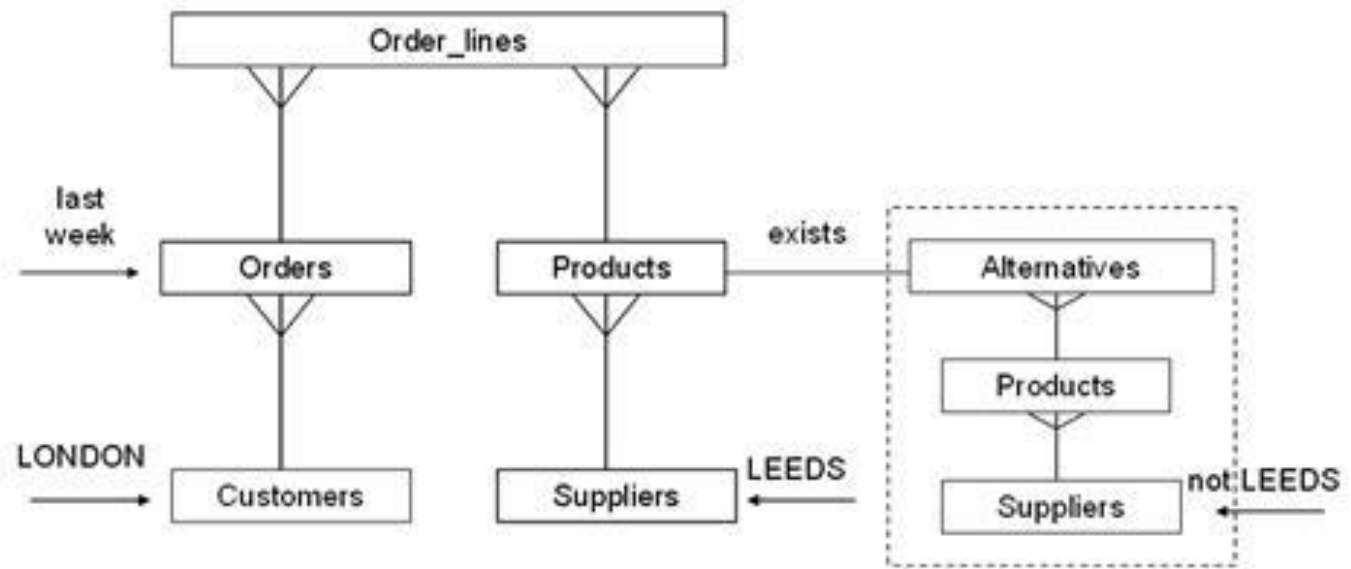
■ Ví dụ:



Các bước viết truy vấn

- Biểu diễn trực quan quan hệ giữa các nguồn dữ liệu (các bảng) tham gia vào truy vấn
 - Các truy vấn dữ liệu được biểu diễn dưới dạng các sơ đồ liên kết giữa các bảng giúp quan sát dễ dàng nguồn dữ liệu và các mệnh đề điều kiện của truy vấn
 - Ví dụ: "Hãy lập báo cáo danh sách **đơn đặt hàng** *trong tuần qua* bởi **khách hàng** *có trụ sở tại LONDON*, cho **các sản phẩm** được cung cấp bởi các **nhà sản xuất** *có trụ sở tại LEEDS* mà có thể *đã được cung cấp từ một địa điểm thay thế*"

Có thể chưa đúng ngay lần đầu, vẽ thử một vài lần

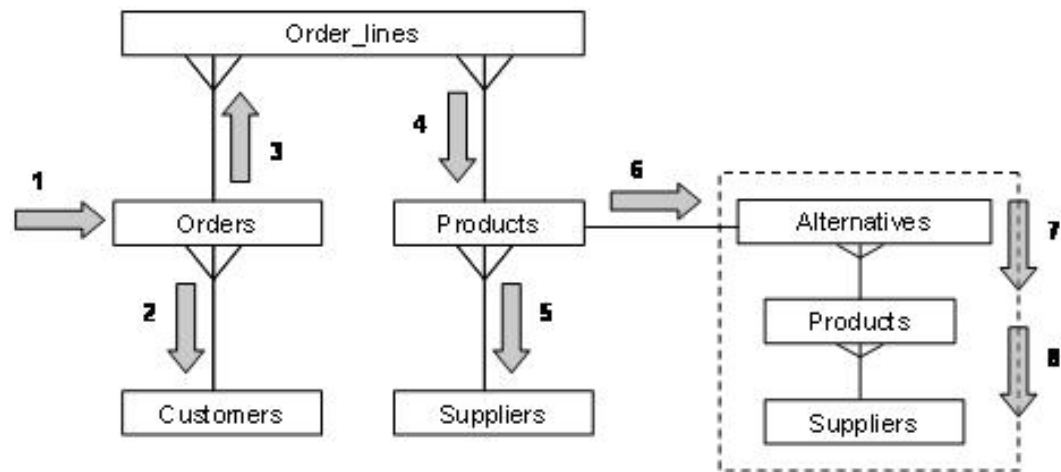


Các bước viết truy vấn

indexes considering?

■ Biểu diễn trực quan

Từ sơ đồ
các bảng đã
phác thảo
xác định
thứ tự các
bảng truy vấn



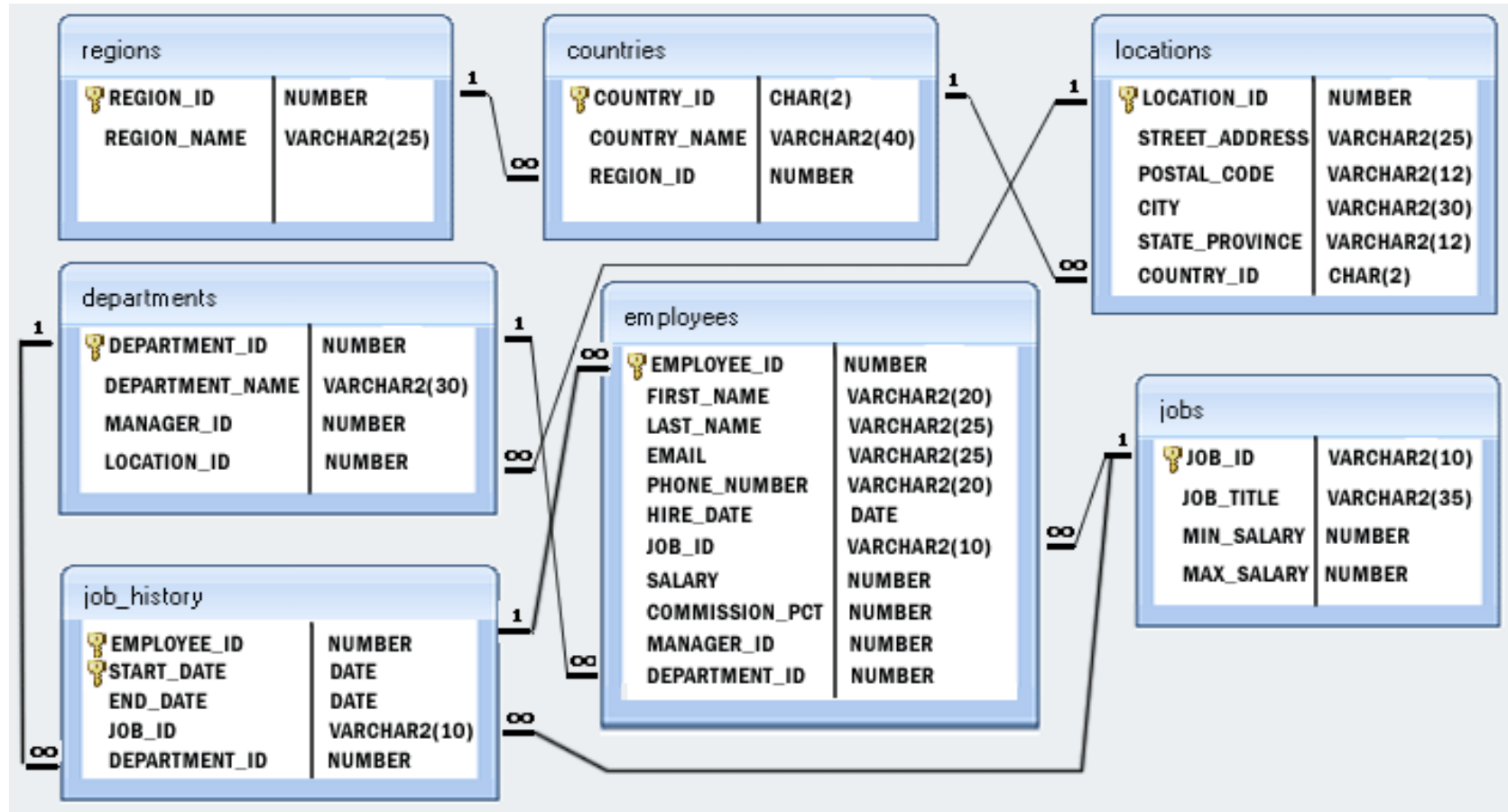
Viết truy vấn
SQL



```
SELECT {list of columns}
FROM      customers cus
         INNER JOIN orders ord ON ord.id_customer = cus.id
         INNER JOIN order_lines orl ON orl.id_order = ord.id
         INNER JOIN products prd1 ON prd1.id = orl.id_product
         INNER JOIN suppliers sup1 ON sup1.id =
                                prd1.id_supplier
WHERE     cus.location = 'LONDON'
         AND ord.date_placed BETWEEN
                                dateadd(day, -7, getdate())
                                AND getdate()
         AND sup1.location = 'LEEDS'
         AND EXISTS ( SELECT NULL
                     FROM alternatives alt
                     INNER JOIN products prd2
                         ON prd2.id = alt.id_product_sub
                     INNER JOIN suppliers sup2
                         ON sup2.id = prd2.id_supplier
                     WHERE alt.id_product = prd1.id
                         AND sup2.location != 'LEEDS' )
```


Các bước viết truy vấn

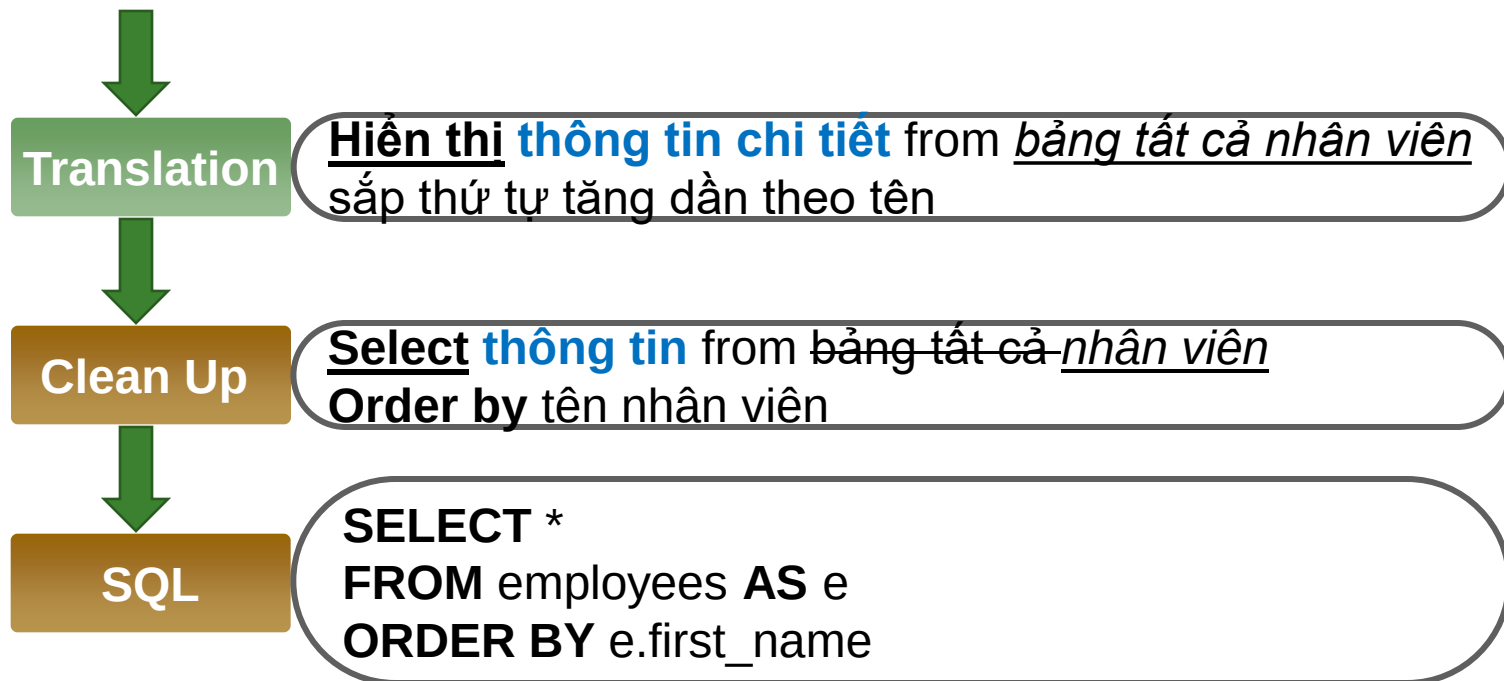
- Ví dụ: Xét cơ sở dữ liệu sau



Các bước viết truy vấn

■ Ví dụ 1

- "Hiển thị thông tin chi tiết của tất cả nhân viên, sắp xếp theo thứ tự tăng dần của tên nhân viên"



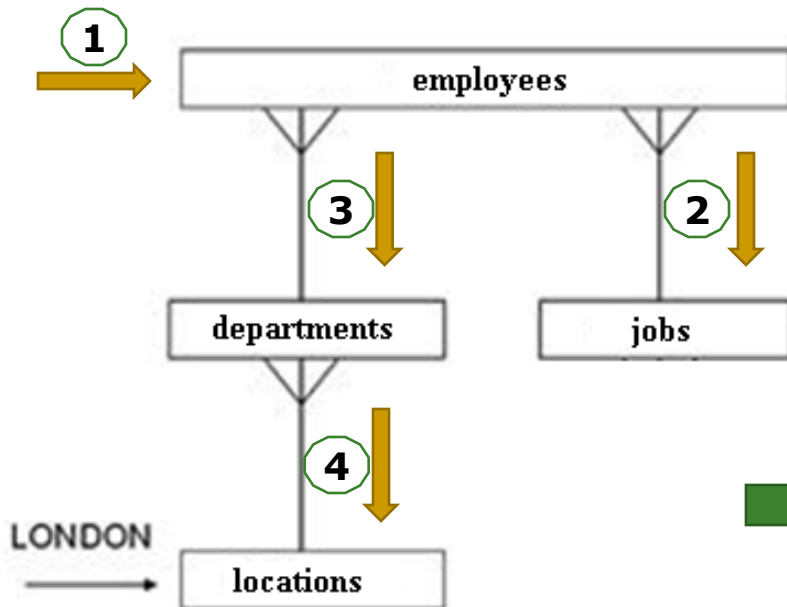
Các bước viết truy vấn

■ Ví dụ 2

- "Tìm các thông tin họ tên, vị trí công việc, tên phòng ban của các nhân viên đang làm việc tại LONDON"

Translation

Hiển thị họ tên, vị trí công việc, tên phòng ban của các nhân viên đang làm việc tại LONDON



```
1 SELECT e.first_name, e.last_name,
2 j.job_title, d.department_name
3 FROM employees AS e
4
5 JOIN jobs AS j
6 ON (e.job_id = j.job_id)
7
8 JOIN departments AS d
9 ON (e.department_id = d.department_id)
10
11 JOIN locations AS l
12 ON (d.location_id = l.location_id)
13
14 WHERE LOWER(l.city) = 'London';
```

Các bước viết truy vấn

■ Ví dụ 3

- "Thay đổi email của các nhân viên làm trong phòng 'Kế toán' thành giá trị 'not available'"

Translation

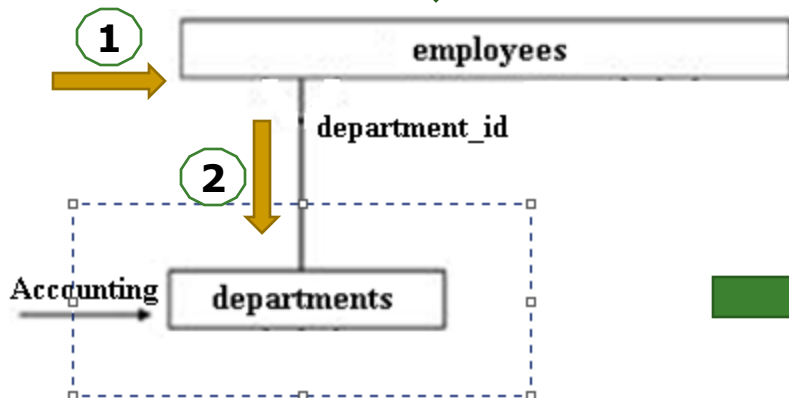
Thay đổi email

của các nhân viên làm việc trong phòng 'kế toán' thành giá trị 'not available'

Clean Up

Update nhân viên

Thiết lập email thành giá trị 'not available' cho các nhân viên của phòng có tên là 'Accounting'



```
1  UPDATE employees
2
3  SET email='not available'
4
5  WHERE department_id =
6      (
7      SELECT department_id
8      FROM departments
9      WHERE department_name='Accounting'
10     );
```

Nội dung

1. Các bước viết truy vấn dữ liệu

 **2. Xây dựng điều kiện lọc dữ liệu**

3. Gom nhóm dữ liệu và các hàm tập hợp

4. Truy vấn trên nhiều bảng dữ liệu

5. Truy vấn con (subquery)

Mệnh đề FROM

- Mệnh đề FROM

- **FROM <list of tables>**
- Thực hiện phép tích của đại số quan hệ, dùng để chỉ ra các bảng chứa dữ liệu cần lấy ra.
- List of tables là danh sách các bảng được phân cách nhau bởi dấu phẩy.
- Mỗi bảng có thể có bí danh bảng đứng ngay phía sau tên bảng được phân cách bởi ký tự trống; khi đó, chỉ sử dụng bí danh bảng và không được sử dụng tên bảng.

Mệnh đề ORDER BY

■ Mệnh đề ORDER BY

- **ORDER BY <list of expressions>**
- Thông thường, thứ tự của các hàng được trả về trong kết quả của truy vấn là không xác định. Mệnh đề ORDER BY có thể được dùng để sắp thứ tự các hàng này.
- Mệnh đề ORDER BY luôn luôn là mệnh đề cuối cùng của lệnh SELECT.
- Thứ tự ngầm định là tăng dần (ASC → ASCending), từ khóa DESC (descending) đứng ngay sau tên cột dùng để chỉ định thứ tự giảm dần.
- Các cột trong mệnh đề ORDER BY không bắt buộc phải có trong mệnh đề SELECT.

Mệnh đề WHERE

■ Mệnh đề WHERE

- **WHERE <row conditions>**
- Thực hiện các phép chọn, phép kết của đại số quan hệ.
- Row conditions là các điều kiện được xét trên mỗi hàng (kết quả điều kiện: True / False), các hàng nào thỏa mãn các điều kiện này thì được đưa vào kết quả của truy vấn.

Mệnh đề WHERE

■ Mệnh đề WHERE

Các phép toán so sánh

=	Bằng
<	Nhỏ hơn
<=	Nhỏ hơn hoặc bằng
>	Lớn hơn
>=	Lớn hơn hoặc bằng
<>	Không bằng
!=	Không bằng
BETWEEN ... AND ...	Giữa hai giá trị
IN (list)	Một trong các giá trị của danh sách
LIKE string	Giống khuôn mẫu
IS NULL	Là giá trị rỗng

%: thay cho *một đoạn ký tự tùy ý*
_: thay cho *một ký tự tùy ý*

Các phép toán phủ định

NOT BETWEEN ... AND ...
NOT IN (list)
NOT LIKE string
IS NOT NULL

Thực hành thiết kế các truy vấn

- 1. Hiển thị thông tin chi tiết tất cả các nhân viên sắp xếp tăng dần theo tên nhân viên
- 2. Lập danh sách họ và tên, ngày sinh, giới tính của các nhân viên nam có lương từ 20000 trở lên
- 3. Tìm thông tin của các nhân viên có tên bắt đầu bằng H
- 4. Liệt kê các nhân viên sinh tháng 11
- 5. Tìm tên và ngày sinh của những nhân viên không có người quản lý trực tiếp

Nội dung

1. Các bước viết truy vấn dữ liệu

2. Xây dựng điều kiện lọc dữ liệu



3. Gom nhóm dữ liệu và các hàm tập hợp

4. Truy vấn trên nhiều bảng dữ liệu

5. Truy vấn con (subquery)

Mệnh đề GROUB BY

■ Mệnh đề GROUP BY

- **GROUP BY <list of expressions>**
- Dùng để phân chia các hàng của một bảng thành các nhóm nhỏ hơn.
- Các hàm nhóm có thể được sử dụng để trả về thông tin chung cho mỗi nhóm.
- Mỗi nhóm chỉ xuất hiện ở một hàng trong kết quả của truy vấn.
- Trong trường hợp lệnh SELECT có cả hai mệnh đề WHERE và GROUP BY thì các hàng sẽ được chọn bởi điều kiện của mệnh đề WHERE, rồi sau đó phân chia các hàng được chọn thành các nhóm.

Mệnh đề GROUB BY

■ Mệnh đề GROUP BY

- Các cột trong mệnh đề GROUP BY không bắt buộc phải có trong mệnh đề SELECT. Để cho kết quả của truy vấn có nghĩa thì các cột của mệnh đề GROUP BY nên có trong mệnh đề SELECT.
- Khi có mệnh đề GROUP BY, tất cả các cột có trong mệnh đề SELECT phải có trong mệnh đề GROUP BY, ngoại trừ chúng ở trong hàm nhóm.
- Nếu lệnh SELECT không có mệnh đề GROUP BY thì toàn bộ bảng được xem là một nhóm. Nếu mệnh đề SELECT có chứa hàm nhóm thì không thể lấy được chi tiết của mỗi hàng của nhóm.

Mệnh đề GROUP BY

- Mệnh đề GROUP BY - Các hàm tập hợp (*aggregate function*) còn được gọi là hàm nhóm (*group function*)

Hàm	Giá trị trả về
AVG ([DISTINCT <u>ALL</u>] n)	Giá trị trung bình của <i>n</i> , bỏ qua các giá trị rỗng.
COUNT ([DISTINCT <u>ALL</u>] expr *)	Số hàng mà <i>expr</i> có giá trị khác rỗng. * làm cho COUNT đếm tất cả các hàng được chọn, bao gồm các hàng trùng nhau và các hàng có giá trị rỗng.
MAX ([DISTINCT <u>ALL</u>] expr)	Giá trị lớn nhất của <i>expr</i> .
MIN ([DISTINCT <u>ALL</u>] expr)	Giá trị nhỏ nhất của <i>expr</i> .
SUM ([DISTINCT <u>ALL</u>] n)	Tổng giá trị của <i>n</i> , bỏ qua các giá trị rỗng.
STDDEV ([DISTINCT <u>ALL</u>] n)	Độ lệch chuẩn (<i>STanDard DEViation</i>) của <i>n</i> , bỏ qua các giá trị rỗng.
VARIANCE ([DISTINCT <u>ALL</u>] n)	Phương sai của <i>n</i> , bỏ qua các giá trị rỗng.

- Lưu ý:
 - Các hàm tập hợp không tính toán các giá trị NULL
 - Chỉ được xuất hiện trong mệnh đề SELECT hoặc HAVING

Thực hành thiết kế các truy vấn

- 6. Đếm số nhân viên trong công ty
- 7. Đếm số nhân viên trong công ty theo từng mã phòng ban
- 8. Xác định lương trung bình của nhân viên theo phòng ban
- 9. Với mỗi phòng, cho biết số dự án phòng đó điều phối
 - Hướng dẫn: `COUNT(expr)`

```
select MaPhong, count(MaDA) as 'So du an'  
from DUAN  
group by MaPhong
```

DUAN	TenDA	MaDA	Diadiem	Phong
	Sản phẩm X	1	Tân Bình	5
	Sản phẩm Y	2	Thủ Đức	5
	Sản phẩm Z	3	Phú Nhuận	5
	Tin học hóa	10	Gò Vấp	4
	Phúc lợi	30	Gò Vấp	4
	Tái tổ chức	20	Phú Nhuận	1

Phong	So du an
5	3
4	2
1	1

Mệnh đề HAVING

■ Mệnh đề HAVING

- **HAVING <group conditions>**
- Dùng để xác định các nhóm được đưa vào kết quả của truy vấn.
- Group conditions là các điều kiện được xét cho mỗi nhóm. Có thể dùng các toán tử tương tự WHERE.
- Mệnh đề HAVING có thể đứng trước mệnh đề GROUP BY, nhưng mệnh đề GROUP BY nên đứng trước để cho dễ hiểu.
- Tất cả các cột có trong mệnh đề HAVING phải có trong mệnh đề GROUP BY, ngoại trừ chúng ở trong hàm nhóm.

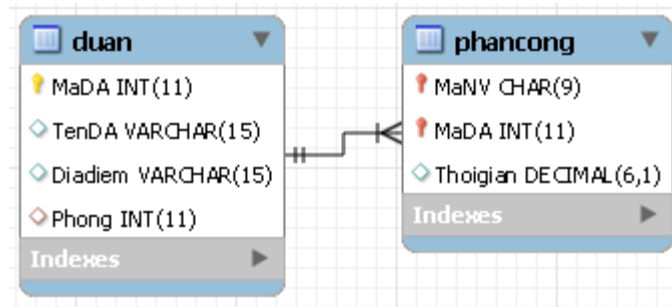
Mệnh đề HAVING

■ Mệnh đề HAVING

- Các nhóm được tạo ra và các hàm nhóm được tính toán trước khi thực hiện mệnh đề HAVING để chọn ra các nhóm.
- Mệnh đề WHERE không được dùng để chọn các nhóm.
- Nếu lệnh SELECT có các mệnh đề WHERE, GROUP BY và HAVING thì thứ tự thực hiện của các mệnh đề này là:
 - (1) WHERE (để chọn các hàng) → (2) kế tiếp GROUP BY (để phân chia nhóm) → (3) sau cùng là HAVING (để chọn các nhóm).

Thực hành thiết kế các truy vấn

- 10. Cho biết **mã số**, **tên dự án** và **số nhân viên** tham gia đối với những **dự án** có nhiều hơn 2 nhân viên tham gia.



```
select d.MaDuAn, d.TenDuAa, count(*) as 'So nhan vien'
from duan d, phancong pc
where d.MaDuAn = pc.MaDuAn
group by d.MaDuAn, d.TenDuAn
having count(*) > 2;
```

- 11. Liệt kê các mã phòng mà có số nhân viên tương ứng ít nhất là 3 nhân viên
- 12. Liệt kê các mã phòng mà có lương trung bình của nhân viên trong mã phòng đó từ 16000 trở lên

Nội dung

1. Các bước viết truy vấn dữ liệu
2. Xây dựng điều kiện lọc dữ liệu
3. Gom nhóm dữ liệu và các hàm tập hợp
- 4. Truy vấn trên nhiều bảng dữ liệu
5. Truy vấn con (subquery)

Mệnh đề JOIN

■ Mệnh đề JOIN

- Mệnh đề join dùng để liên kết dữ liệu từ nhiều hơn 1 bảng theo điều kiện kết nối.
- Tích đề các
 - table **STRAIGHT_JOIN** table
- Inner Join
 - table [**INNER** | CROSS] **JOIN** table [join_condition]
- Left Join
 - table_reference **LEFT** [OUTER] **JOIN** table_reference [join_condition]
- Right Join
 - table_reference **RIGHT** [OUTER] **JOIN** table_reference [join_condition]

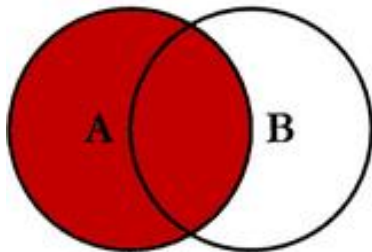
Mệnh đề JOIN

- **Tích Cartesian**: việc kết nối nhiều bảng mà không có điều kiện kết nối trong mệnh đề ON sẽ tạo ra tích cartesian giữa 2 bảng.
- **Inner join**: dữ liệu từ nhiều bảng được hiển thị sau khi so sánh giá trị trong 1 cột chung. Chỉ những hàng mà có giá trị thoả mãn điều kiện kết nối trong cột chung đó mới được hiển thị.
- Có thể thực hiện kết nối 2 bảng với điều kiện kết nối dùng toán tử không bằng nhau.
- **Outer join**: được dùng để cho ra kết quả chứa tất cả các hàng của 1 bảng và các hàng trùng nhau của bảng còn lại. Những cột mà không có giá trị phù hợp sẽ được hiển thị giá trị NULL.

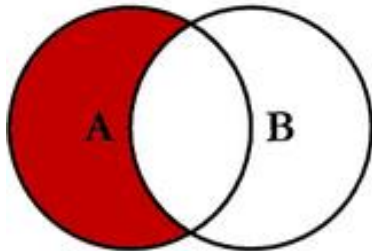
Mệnh đề JOIN

■ Minh họa các loại JOIN

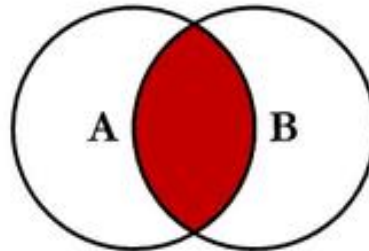
SQL JOINS



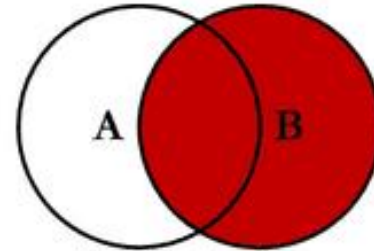
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key
```



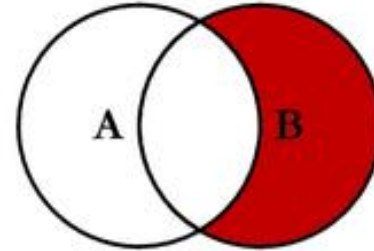
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key  
WHERE B.Key IS NULL
```



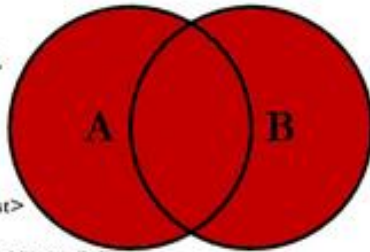
```
SELECT <select_list>  
FROM TableA A  
INNER JOIN TableB B  
ON A.Key = B.Key
```



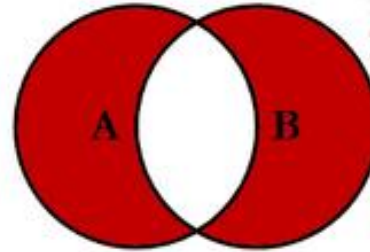
```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL  
OR B.Key IS NULL
```

Mệnh đề JOIN

- Khi kết nối các bảng sử dụng JOIN:
 - Kết nối chỉ tồn tại trong thời gian truy vấn.
 - Kết nối không thay đổi dữ liệu trong các bảng của cơ sở dữ liệu.
 - Nên tạo bí danh (alias) cho tên bảng để tránh gõ tên dài và làm truy vấn dễ đọc hơn.
 - Nếu kết nối nhiều hơn 2 bảng thì kết nối 2 bảng trước, sau đó kết nối nhóm này với bảng thứ ba.

Thực hành thiết kế các truy vấn

- 13. Hiển thị tên nhân viên và tên phòng ban mà nhân viên đó đang làm việc.
- 14. Lập danh sách tất cả các nhân viên và tên của phòng ban mà nhân viên đó là trưởng phòng nếu có.
- 15. Liệt kê tên tất cả các nhân viên và tên của người quản lý trực tiếp nhân viên đó nếu có

Mệnh đề UNION

■ Mệnh đề UNION:

- UNION thực hiện phép hợp kết quả của hai truy vấn.
- Từ khóa ALL cho phép các hàng trong kết quả có thể trùng nhau.

```
SELECT ...  
    UNION [ALL | DISTINCT]  
SELECT ...  
    [UNION [ALL | DISTINCT]  
    [SELECT ...]
```

■ Chú ý:

- Chỉ sử dụng 1 loại UNION ALL hoặc UNION DISTINCT trong 1 câu lệnh UNION
- Số lượng, thứ tự các cột phải giống nhau trong mọi truy vấn
- Kiểu dữ liệu của các cột tương ứng trong các truy vấn phải tương thích

Mệnh đề UNION

- Ví dụ: kết quả của các truy vấn sau là gì?

```
CREATE TABLE t1 (id INT);
```

```
CREATE TABLE t2 (id INT);
```

```
INSERT INTO t1 VALUES (1), (1), (1), (2), (2);
```

```
INSERT INTO t2 VALUES (2), (2), (3), (3), (3);
```

```
(SELECT * FROM t1)
```

```
UNION DISTINCT
```

```
(SELECT * FROM t2);
```

```
(SELECT * FROM t1 ORDER BY id)
```

```
UNION ALL
```

```
(SELECT * FROM t2 ORDER BY id);
```

Thực hành thiết kế các truy vấn

- 16. Chuẩn bị cho kỳ du lịch hè, công ty cần lập danh sách thông tin về nhân viên và các thân nhân của nhân viên để gửi cho bên tour, danh sách cần có: tên, ngày sinh, giới tính hãy xây dựng truy vấn lấy các dữ liệu này

Nội dung

1. Các bước viết truy vấn dữ liệu
2. Xây dựng điều kiện lọc dữ liệu
3. Gom nhóm dữ liệu và các hàm tập hợp
4. Truy vấn trên nhiều bảng dữ liệu
5. Truy vấn con (subquery)

Truy vấn con (subquery)

■ Truy vấn con (subquery)

- Là một truy vấn (lệnh SELECT) nằm trong một truy vấn khác.
- Truy vấn ngoài (outer query)
- Truy vấn trong (inner query)
- Truy vấn chính (main query)

The diagram illustrates a SQL query structure. The text **OUTER QUERY** is positioned to the left of a yellow bracket that groups the following SQL clauses: **SELECT ***, **FROM t1**, and **WHERE column1 = (**. The closing parenthesis **)** is located below the bracket. To the right of the opening parenthesis, the text **SUBQUERY** is positioned to the right of another yellow bracket that groups the clauses **SELECT column1** and **FROM t2**. The query concludes with a semicolon **;**.

```
SELECT *  
FROM t1  
WHERE column1 = (  
    SELECT column1  
    FROM t2  
);
```

■ Vị trí xuất hiện

- Trong điều kiện của mệnh đề WHERE.
- Như là một bảng trong mệnh đề FROM.
- Trong điều kiện của mệnh đề HAVING.

Truy vấn con (subquery)

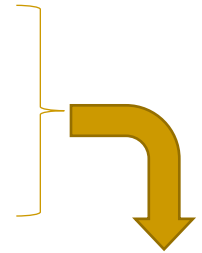
■ Các loại truy vấn con

- Truy vấn con lồng nhau (nested subquery)
 - Không phụ thuộc vào dữ liệu của truy vấn ngoài.
 - Được thực hiện duy nhất một lần trước khi thực hiện truy vấn ngoài.
 - Kết quả của truy vấn con được dùng để thực hiện truy vấn ngoài.
- Truy vấn con tương quan (correlated subquery)
 - Sử dụng dữ liệu của truy vấn ngoài.
 - Đối với mỗi hàng của truy vấn ngoài, dữ liệu của hàng này được dùng để thực hiện truy vấn con, kết quả của truy vấn con được dùng để thực hiện truy vấn ngoài.

Truy vấn con lồng nhau

- Ví dụ 1: Cho biết các nhân viên cùng phòng với nhân viên "Nguyễn Văn A"?

```
SELECT    n.TenNhanVien, n.MaNhanVien
FROM      nhanvien n
WHERE     n.MaPhong IN (
          SELECT n2.MaPhong
          FROM  nhanvien n2
          WHERE n2.TenNhanVien = 'Nguyễn Văn A'
        ) ;
```



- Truy vấn con thực hiện trước xác định danh sách các giá trị **Phong**
- Các bản ghi trong truy vấn cha sẽ kiểm tra điều kiện với tập giá trị này

Bảng NHANVIEN ở truy vấn con không liên quan đến bảng NHANVIEN ở truy vấn cha

Truy vấn con lồng nhau

- Ví dụ 2: Đưa ra danh sách tên và mã số các nhân viên làm việc ở phòng kinh doanh hoặc kỹ thuật?

```
SELECT    n.TenNhanVien, n.MaNhanVien
FROM      nhanvien n
WHERE     n.Phong IN (
          SELECT p.MaPhong
          FROM phongban p
          WHERE p.TenPhong IN ('Kinh doanh', 'Ky
                                thuat')
        ) ;
```

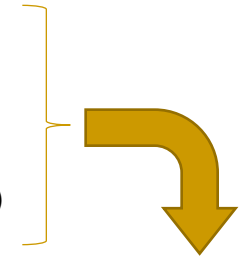
- Ví dụ 3: Cho biết họ tên của các nhân viên có lương cao nhất trong công ty?

```
SELECT    n.HoNhanVien, n.TenNhanVien
FROM      nhanvien n
WHERE     n.Luong = (
          SELECT max(n2.Luong)
          FROM nhanvien n2
        ) ;
```


Truy vấn con tương quan

- Ví dụ 4: Tìm những nhân viên không có thân nhân nào?

```
SELECT    n.TenNhanVien, n.MaNhanVien
FROM      nhanvien n
WHERE     NOT EXISTS (
    SELECT *
    FROM    thannhan tn
    WHERE   tn.MaNhanVien = n.MaNhanVien)
);
```



Trong truy vấn con này có tham chiếu đến thuộc tính MaNV của bảng NHANVIEN trên truy vấn cha

- Với mỗi dòng trong bảng **nhanvien** thì truy vấn con trên bảng **thannhan** phải thực hiện 1 lần

Thực hành thiết kế các truy vấn

- 17. Thống kê mỗi phòng ban có bao nhiêu nhân viên
- 18. Đưa ra tên phòng ban có nhân viên hưởng lương cao nhất

Truy vấn con với ANY, IN, SOME, ALL, EXISTS

■ Cú pháp tổng quát

- <tên thuộc tính> **IN** <truy vấn con>
- <tên thuộc tính> <phép toán so sánh> **ALL** <truy vấn con>
- <tên thuộc tính> <phép toán so sánh> **ANY** <truy vấn con>
- **EXISTS** <truy vấn con>

■ Phép toán

- **IN** - kiểm tra sự tồn tại của một giá trị trong một tập hợp.
- **ALL** - so sánh một giá trị với tất cả các giá trị của tập hợp.
- **ANY** / **SOME** - so sánh một giá trị với một giá trị nào đó của tập hợp.
- **ALL**, **ANY**, **SOME** được kết hợp với các phép toán so sánh {=, <, ≤, >, ≥, <>}.
- **EXISTS** - kiểm tra sự tồn tại của kết quả của một câu truy vấn.

Truy vấn con với ANY, IN, SOME, ALL, EXISTS

■ Lượng từ **ALL**

- Where <Tên thuộc tính> <Phép so sánh> **ALL** (<Tập hợp giá trị | truy vấn con>)
 - **TRUE**: khi **một giá trị** của thuộc tính thỏa mãn **phép so sánh** với **mọi phần tử** trong tập hợp giá trị (hoặc tập giá trị do truy vấn con trả về).
 - **FALSE**: Ngược lại

■ Ví dụ: Lập danh sách tên các nhân viên có lương lớn hơn lương của tất cả các nhân viên phòng Quan ly?

```
SELECT    n.HoNhanVien, n.TenNhanVien
FROM      nhanvien n
WHERE     n.Luong >= all (
          SELECT n1.Luong
          FROM  nhanvien n1 inner join phongban p
              on n1.MaPhong = p.MaPhong
          WHERE p.TenPhong = 'Quan ly'
          ) ;
```

Thực hành: Đưa ra tên các phòng ban có nhiều nhân viên nhất?

Truy vấn con với ANY, IN, SOME, ALL, EXISTS

■ Lượng từ **any | some**

- **WHERE** <Tên thuộc tính> <Phép so sánh> **any | some** (<Tập giá trị| Truy vấn con>)
 - **TRUE**: khi **một giá trị** của thuộc tính thỏa mãn **phép so sánh** với **ít nhất một phần tử** trong tập hợp giá trị (hoặc tập giá trị do truy vấn con trả về).
 - **FALSE**: Ngược lại

■ Ví dụ: Cho viết tên các nhân viên có lương lớn hơn lương của ít nhất một nhân viên phòng Quan lý?

```
SELECT    n.HoNhanVien, n.TenNhanVien
FROM      nhanvien n
WHERE     n.Luong >= any (
    SELECT n1.Luong
    FROM   nhanvien n1 inner join phongban p
    on     n1.MaPhong = p. MaPhong
    WHERE  p.TenPhong = 'Quan ly'
);
```

Truy vấn con với ANY, IN, SOME, ALL, EXISTS

■ Lượng từ **Exists** (tồn tại)

- Where **Exists** (<Tập hợp| truy vấn con>)
 - True: nếu tập hợp khác rỗng
 - False: ngược lại

■ Thực hành:

- Đưa ra thông tin các phòng ban có nhân viên
- Đưa ra thông tin các phòng ban không có nhân viên
- Đưa ra thông tin các nhân viên chưa cung cấp thông tin thân nhân
- Đưa ra thông tin các dự án không có nhân viên tham gia

Truy vấn con trong mệnh đề FROM

■ Cú pháp

- **SELECT** ... **FROM** (<subquery>) **AS** <name> ...

■ Ví dụ:

Subquery forms the derived table used in the FROM clause of the outer query

One column of the subquery is an aggregate function that has an alias name. That alias can then be referred to in the outer query

```
SELECT Product_Description, Standard_Price, Avg_Price  
FROM (SELECT AVG(Standard_Price) Avg_Price  
      FROM Product_T), Product_T  
WHERE Standard_Price > Avg_Price;
```

The WHERE clause normally cannot include aggregate functions, but because the aggregate is performed in the subquery its result can be used in the outer query's WHERE clause

